

Guía de Referencia MIPS32

Equivalencias en Lenguaje C y Ejemplos de Uso

Fabian Ochoa

May 2026

1 Aritmética

1.1 ADD Suma con desbordamiento (Add)

¿Para qué sirve?: Suma los valores de dos registros y guarda el resultado en un tercer registro. Si el resultado excede el límite representable (overflow), se genera una excepción de hardware.

ENSAMBLADOR MIPS32	EQUIVALENTE EN C
add \$s1, \$s2, \$s3 # \$s1 = \$s2 + \$s3	a = b + c;

1.2 SUB Resta con desbordamiento (Subtract)

¿Para qué sirve?: Resta el valor del segundo registro operando al primer registro operando, almacenando el resultado en un tercer registro. Genera una excepción en caso de desbordamiento (overflow).

ENSAMBLADOR MIPS32	EQUIVALENTE EN C
sub \$s1, \$s2, \$s3 # \$s1 = \$s2 - \$s3	a = b - c;

1.3 ADDI Suma inmediata con desbordamiento (Add Immediate)

¿Para qué sirve?: Suma el valor de un registro con una constante entera con signo de 16 bits, guardando el resultado en el registro de destino. Controla desbordamientos.

ENSAMBLADOR MIPS32	EQUIVALENTE EN C
addi \$s1, \$s2, 100 # \$s1 = \$s2 + 100	a = b + 100;

2 Transferencia de Datos

2.1 LW Cargar palabra de memoria (Load Word)

¿Para qué sirve?: Lee un dato de 32 bits (una palabra completa) desde una dirección de memoria especificada (base + desplazamiento) y lo almacena en un registro.

ENSAMBLADOR MIPS32	EQUIVALENTE EN C
lw \$s1, 100(\$s2) # \$s1 = Memoria[\$s2 + 100]	a = *(int*)((char*)b + 100);

2.2 SW Almacenar palabra en memoria (Store Word)

¿Para qué sirve?: Copia un valor de 32 bits desde un registro hacia una dirección específica de la memoria RAM.

ENSAMBLADOR MIPS32	EQUIVALENTE EN C
sw \$s1, 100(\$s2) # Memoria[\$s2 + 100] = \$s1	*(int*)((char*)b + 100) = a;

2.3 LH Cargar media palabra con signo (Load Halfword)

¿Para qué sirve?: Lee un dato de 16 bits (media palabra) de la memoria, extiende su signo para rellenar los 32 bits y lo coloca dentro del registro destino.

ENSAMBLADOR MIPS32	EQUIVALENTE EN C
lh \$s1, 100(\$s2) # \$s1 = Memoria[\$s2 + 100]	short temp = *(short*)((char*)b + 100); a = (int)temp;

2.4 SH Almacenar media palabra (Store Halfword)

¿Para qué sirve?: Guarda los 16 bits menos significativos (los más bajos) de un registro en una dirección de memoria.

ENSAMBLADOR MIPS32	EQUIVALENTE EN C
sh \$s1, 100(\$s2) # Memoria[\$s2 + 100] = \$s1	*(short*)((char*)b + 100) = (short)a;

2.5 LB Cargar byte con signo (Load Byte)

¿Para qué sirve?: Lee un único byte (8 bits) desde la memoria, realiza una extensión de signo a 32 bits y lo almacena en el registro.

ENSAMBLADOR MIPS32	EQUIVALENTE EN C
lb \$s1, 100(\$s2) # \$s1 = Memoria[\$s2 + 100]	char temp = *(char*)((char*)b + 100); a = (int)temp;

2.6 SB Almacenar byte (Store Byte)

¿Para qué sirve?: Copia únicamente el byte menos significativo (los 8 bits más bajos) de un registro a una dirección de memoria.

ENSAMBLADOR MIPS32	EQUIVALENTE EN C
sb \$s1, 100(\$s2) # Memoria[\$s2 + 100] = \$s1	*(char*)((char*)b + 100) = (char)a;

2.7 LUI Cargar inmediato superior (Load Upper Immediate)

¿Para qué sirve?: Carga una constante de 16 bits en la parte alta (los 16 bits más significativos) de un registro y rellena los 16 bits inferiores con ceros. Es clave para construir constantes de 32 bits.

ENSAMBLADOR MIPS32	EQUIVALENTE EN C
lui \$s1, 100 # \$s1 = 100 <= 16	a = 100 <= 16;

3 Lógica

3.1 AND Operación AND bit-a-bit (And)

¿Para qué sirve?: Realiza la operación lógica intersección (AND) elemento por elemento binario entre dos registros. Útil para aplicar máscaras y apagar bits específicos.

ENSAMBLADOR MIPS32	EQUIVALENTE EN C
and \$s1, \$s2, \$s3 # \$s1 = \$s2 & \$s3	a = b & c;

3.2 OR Operación OR bit-a-bit (Or)

¿Para qué sirve?: Ejecuta la unión lógica (OR) bit por bit entre los contenidos de dos registros. Se usa frecuentemente para encender u obligar bits a valer 1.

ENSAMBLADOR MIPS32	EQUIVALENTE EN C
or \$s1, \$s2, \$s3 # \$s1 = \$s2 \$s3	a = b c;

3.3 NOR Operación NOR bit-a-bit (Nor)

¿Para qué sirve?: Ejecuta la negación de la unión (NOR) lógica bit a bit de dos registros. MIPS carece de una instrucción “NOT” dedicada, por lo que se utiliza nor con el registro cero (\$zero) para invertir bits.

ENSAMBLADOR MIPS32	EQUIVALENTE EN C
nor \$s1, \$s2, \$s3 # \$s1 = ~(\$s2 \$s3)	a = ~(b c);

3.4 ANDI Operación AND inmediata (And Immediate)

¿Para qué sirve?: Aplica una operación AND bit a bit entre un registro y una constante numérica sin signo de 16 bits.

ENSAMBLADOR MIPS32	EQUIVALENTE EN C
andi \$s1, \$s2, 100 # \$s1 = \$s2 & 100	a = b & 100;

3.5 ORI Operación OR inmediata (Or Immediate)

¿Para qué sirve?: Aplica una operación OR bit a bit entre un registro y una constante numérica sin signo de 16 bits.

ENSAMBLADOR MIPS32	EQUIVALENTE EN C
ori \$s1, \$s2, 100 # \$s1 = \$s2 100	a = b 100;

3.6 SLL Desplazamiento lógico a la izquierda (Shift Left Logical)

¿Para qué sirve?: Desplaza los bits de un registro a la izquierda por el valor entero definido. Los espacios vacíos se rellenan con ceros. Equivale aritméticamente a multiplicar por potencias de 2.

ENSAMBLADOR MIPS32	EQUIVALENTE EN C
sll \$s1, \$s2, 10 # \$s1 = \$s2 <= 10	a = b <= 10;

3.7 SRL Desplazamiento lógico a la derecha (Shift Right Logical)

¿Para qué sirve?: Mueve los bits de un registro hacia la derecha una cantidad fija, introduciendo ceros en el extremo izquierdo. Equivale a dividir una variable sin signo entre potencias de 2.

ENSAMBLADOR MIPS32	EQUIVALENTE EN C
srl \$s1, \$s2, 10 # \$s1 = \$s2 >= 10	a = (unsigned int)b >= 10;

4 Salto Condicional

4.1 BEQ Saltar si son iguales (Branch on Equal)

¿Para qué sirve?: Compara los valores de dos registros. Si los valores son exactamente iguales, el flujo del programa desvía su ejecución saltando hacia una etiqueta determinada del código.

ENSAMBLADOR MIPS32	EQUIVALENTE EN C
beq \$s1, \$s2, L # Si \$s1 == \$s2 saltar a L	if (a == b) { goto L; }

4.2 BNE Saltar si no son iguales (Branch on Not Equal)

¿Para qué sirve?: Compara el contenido de dos registros y desvía la ejecución hacia una etiqueta si los datos almacenados difieren.

ENSAMBLADOR MIPS32	EQUIVALENTE EN C
bne \$s1, \$s2, L # Si \$s1 != \$s2 saltar a L	if (a != b) { goto L; }

4.3 SLT Establecer si es menor que (Set on Less Than)

¿Para qué sirve?: Evalúa si el primer registro es menor que el segundo (con signo). Si es verdadero, coloca un 1 en el registro de destino; si es falso, coloca un 0. Se usa junto a beq/bne para condicionales complejos (<, >).

ENSAMBLADOR MIPS32	EQUIVALENTE EN C
slt \$s1, \$s2, \$s3 # Si \$s2 < \$s3, \$s1=1; sino, \$s1=0	a = (b < c) ? 1 : 0;

4.4 SLTI Establecer si es menor que un inmediato (Set on Less Than Immediate)

¿Para qué sirve?: Compara el contenido de un registro con un número constante con signo. Si el registro es numéricamente menor, el destino se fija en 1; en caso contrario, pasa a valer 0.

ENSAMBLADOR MIPS32	EQUIVALENTE EN C
slti \$s1, \$s2, 100 # Si \$s2 < 100, \$s1=1; sino, \$s1=0	a = (b < 100) ? 1 : 0;

5 Salto Incondicional

5.1 J Salto absoluto (Jump)

¿Para qué sirve?: Fuerza al procesador a saltar directamente y sin condiciones a una dirección del código del programa (representada por una etiqueta).

ENSAMBLADOR MIPS32	EQUIVALENTE EN C
j L # Salta incondicionalmente a la etiqueta L	goto L;

5.2 JR Salto a registro (Jump Register)

¿Para qué sirve?: Salta a la dirección de memoria exacta almacenada dentro de un registro. Su uso principal es regresar de un procedimiento o función usando el registro de retorno de dirección (\$ra).

ENSAMBLADOR MIPS32	EQUIVALENTE EN C
jr \$ra # Salta a la dirección contenida en \$ra	return;

5.3 JAL Saltar y enlazar (Jump and Link)

¿Para qué sirve?: Realiza un salto a una subrutina o función y, de manera automática, guarda la dirección de la siguiente instrucción ($PC + 4$) dentro del registro de enlace (\$ra) para permitir el retorno posterior.

ENSAMBLADOR MIPS32	EQUIVALENTE EN C
jal mi_funcion # Salta a mi_funcion y guarda retorno	mi_funcion();